

Data Structures – Unit 08

Bucharest University of Economic Studies

Heaps and Priority queues



Heap

A heap is a specialized tree-based data structure that satisfies the property:

if **A** is parent for **B** then between the values of **A** and **B** a semantic relation of total order is applied.



Binary-Heap

A Binary-Heap structure is a complete binary tree which can be linearized.



Binary-Heap

If node **A** is **parent** of node **B**, then:

-if $\text{value}(A) \geq \text{value}(B) \Rightarrow$ **max Heap**

-if $\text{value}(A) \leq \text{value}(B) \Rightarrow$ **min Heap**

The maximum or minimum value is always accessible with $O(1)$ complexity time.



Binary-Heap

```
void processing()  
{  
    while(true)  
    {  
        task x = GetNextTask(tasks);  
        process(x);  
    }  
}
```



Binary-Heap

GetTask()

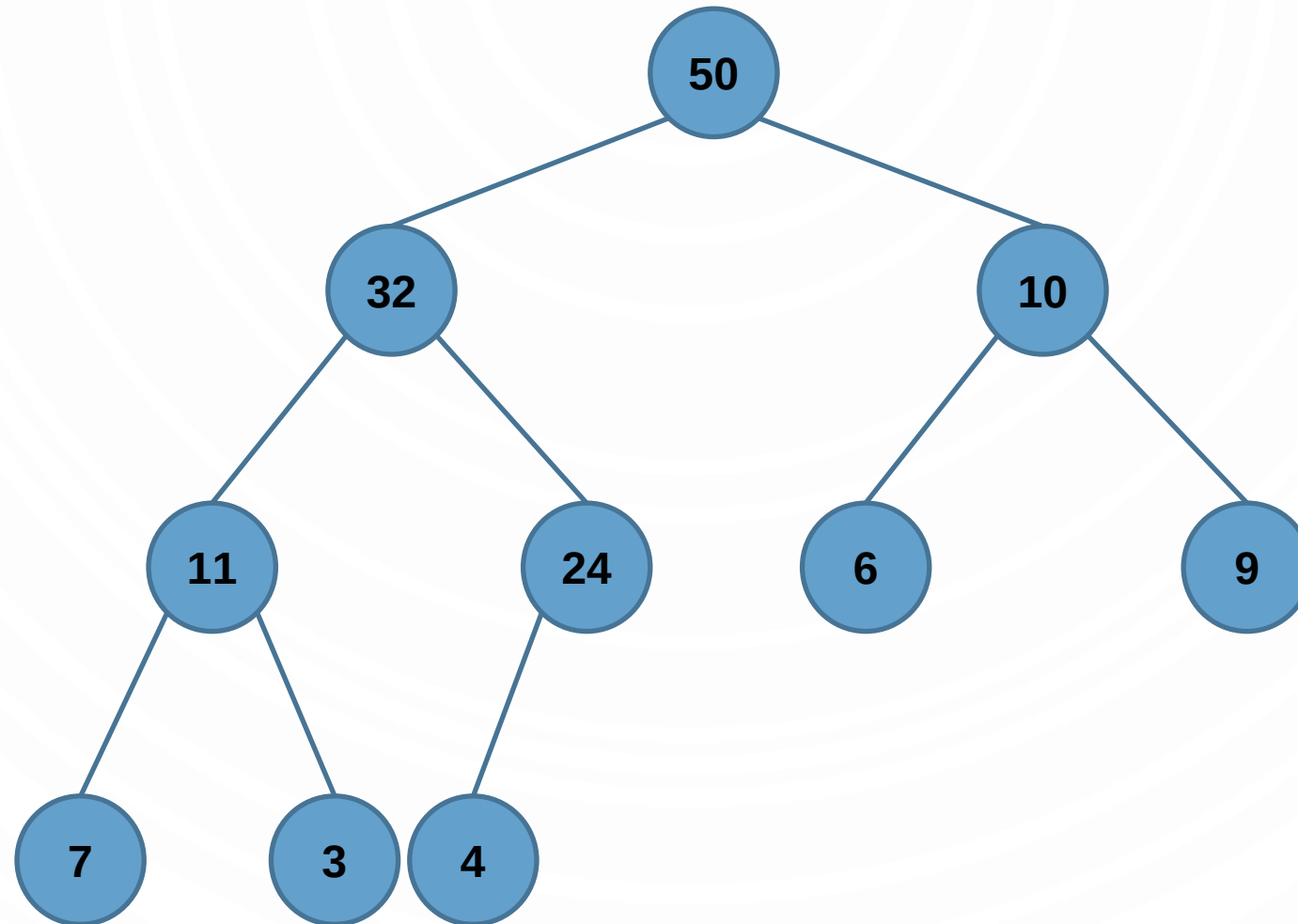
Stacks – the newest task

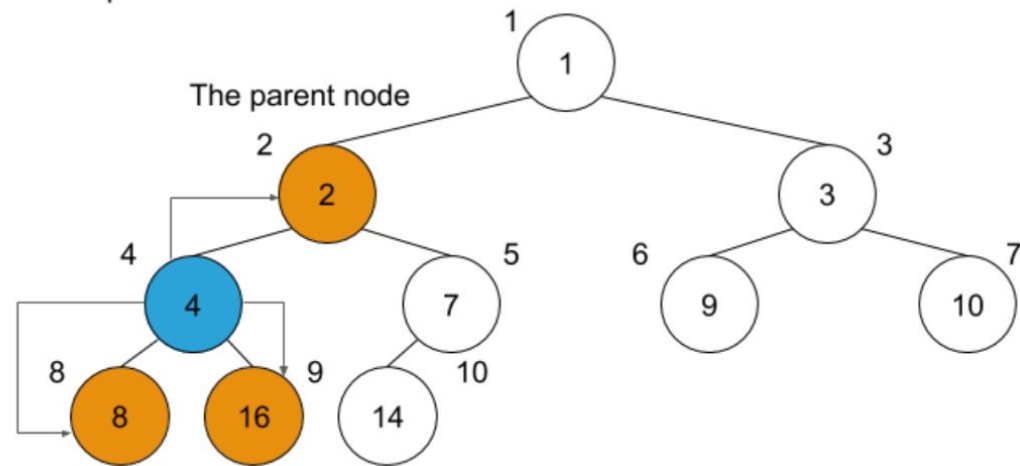
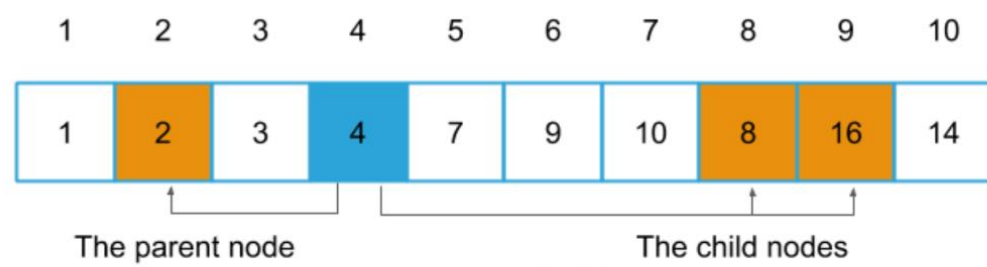
Queues – the oldest task

Priority Queues – the most urgent



Binary-Heap





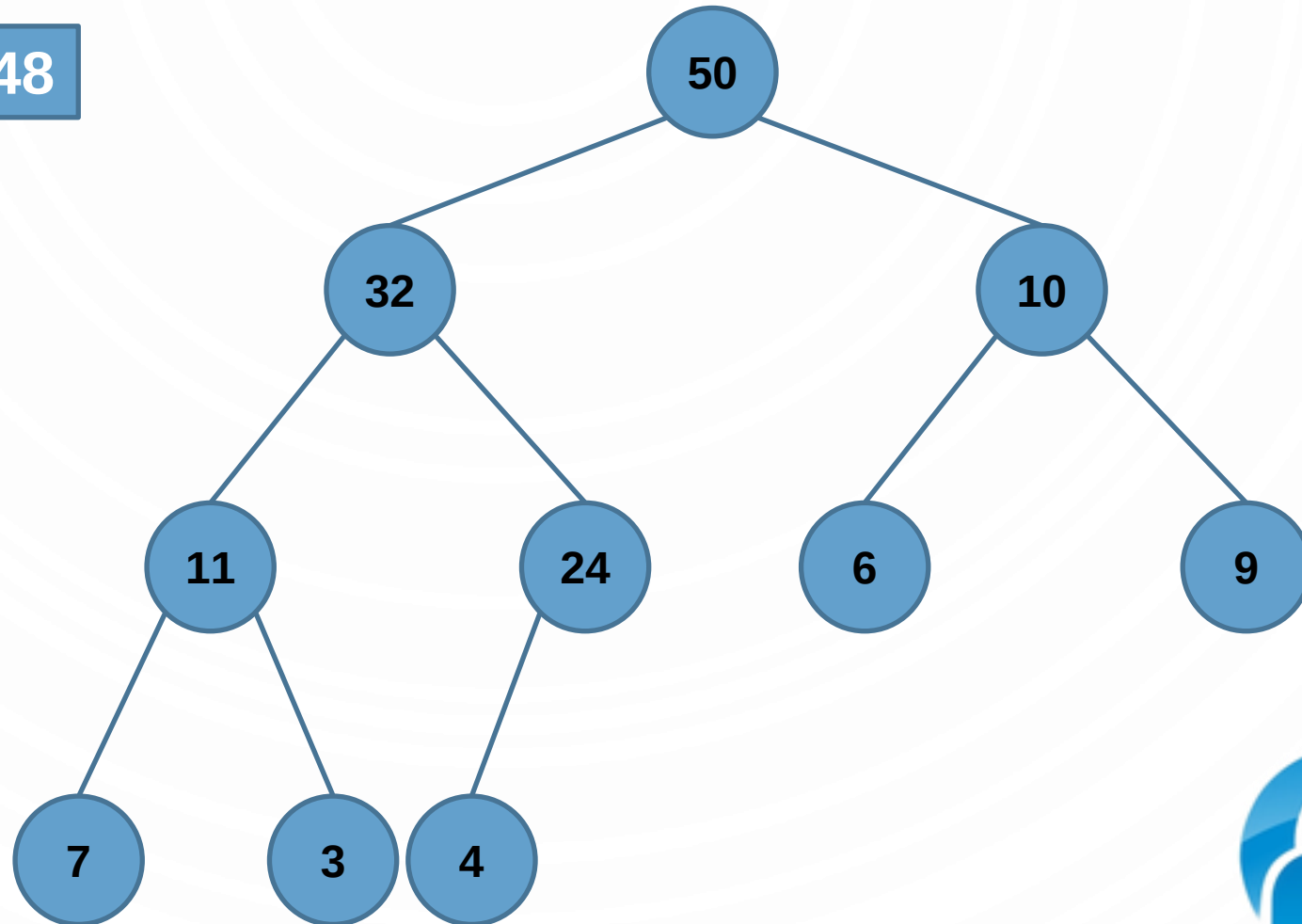
Binary-Heap

$$\text{node } i = \begin{cases} \text{left descendent, } 2i \\ \text{right descendent, } 2i + 1 \\ \text{parent, } \left\lfloor \frac{i}{2} \right\rfloor \end{cases}$$

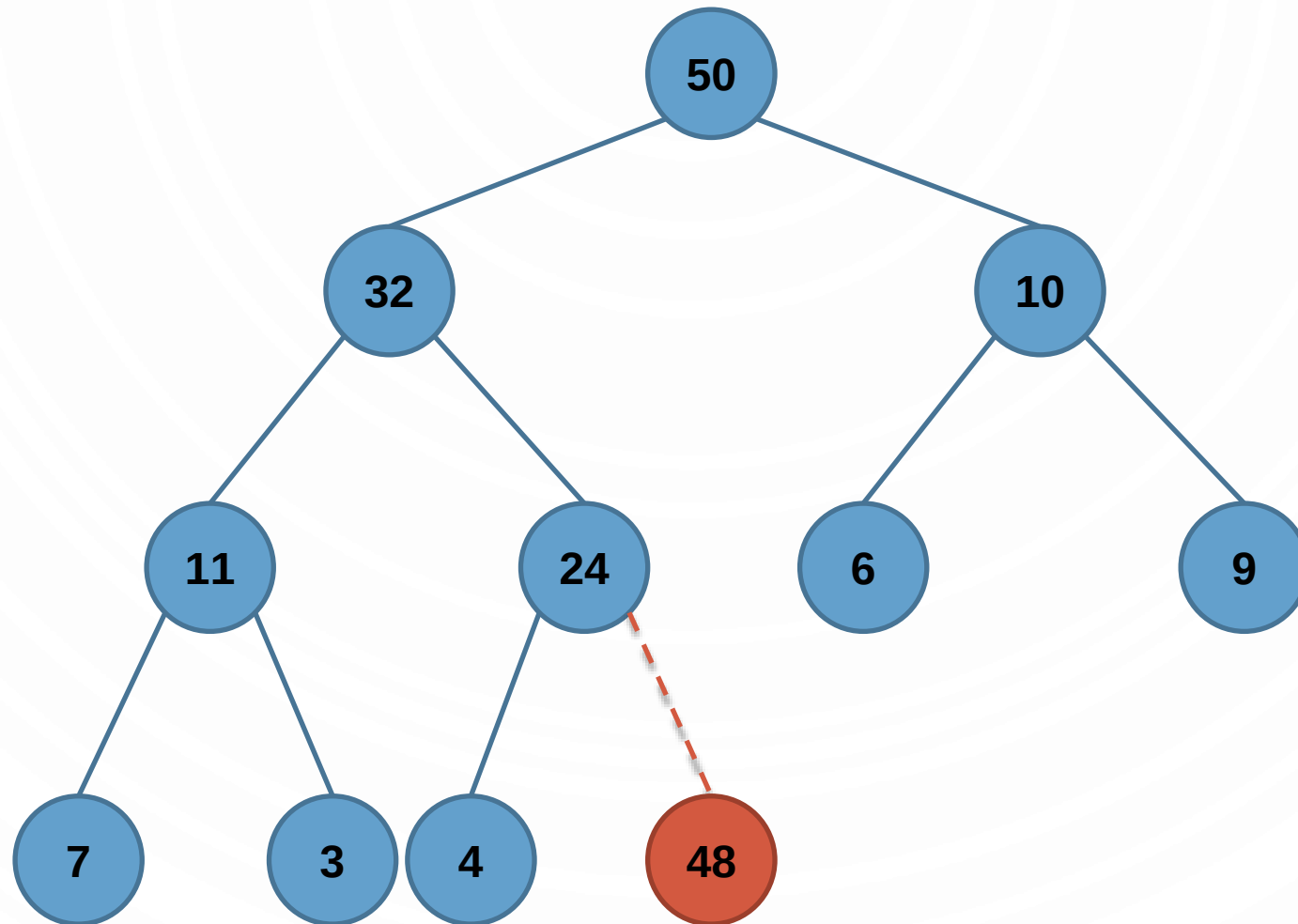


Heap - Insertion

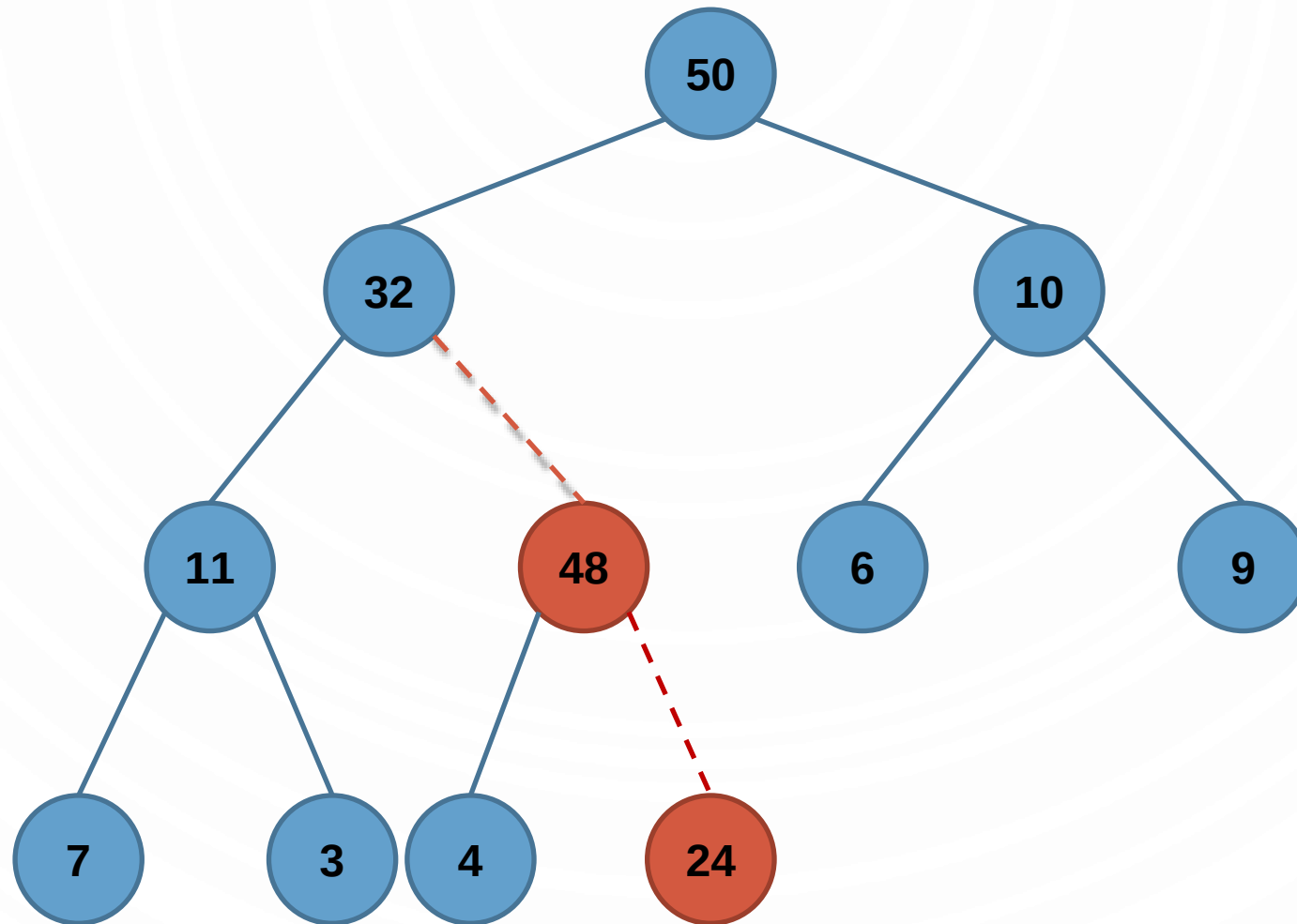
Insert element 48



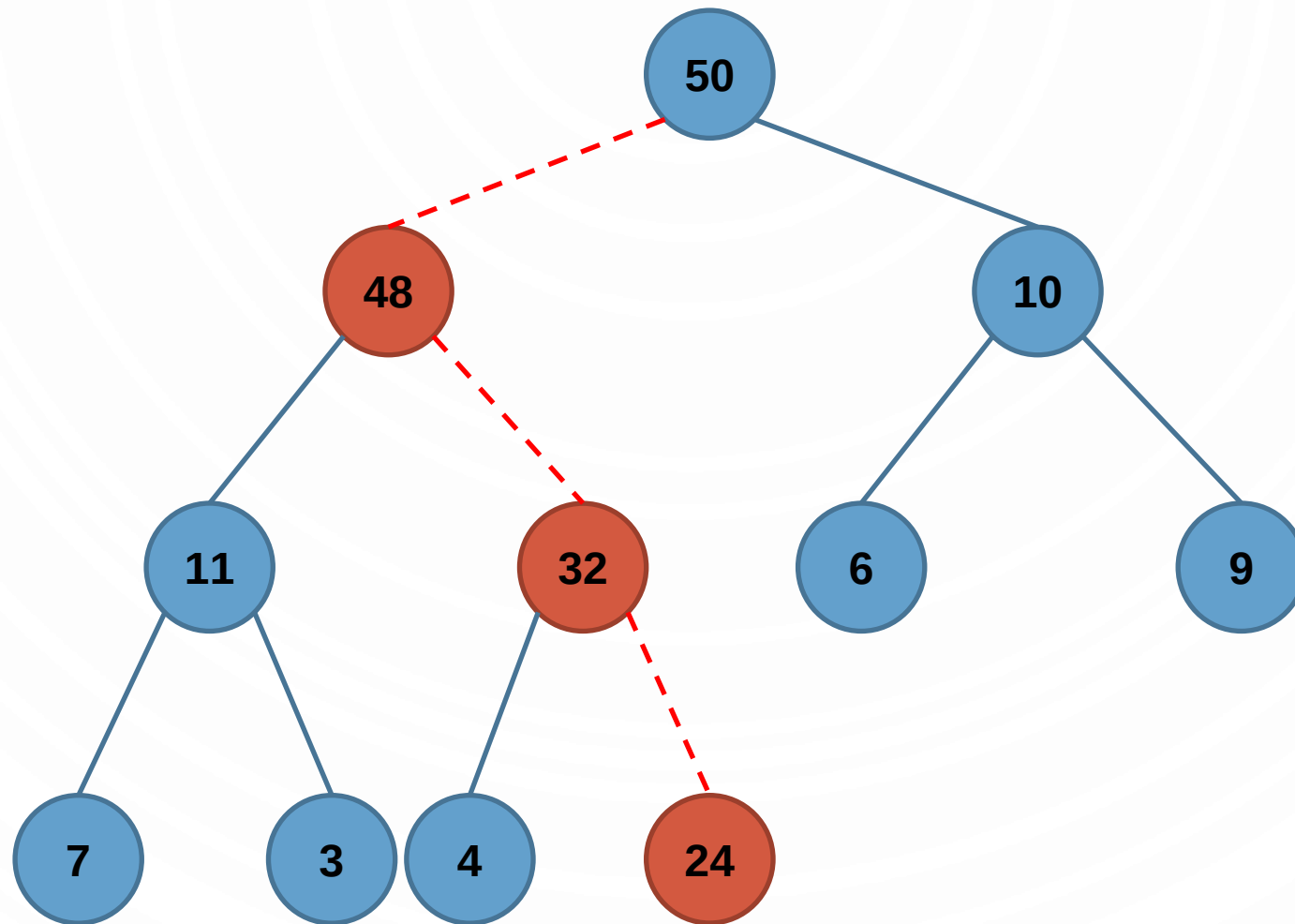
Heap - Insertion



Heap - Insertion



Heap - Insertion



Heap - Insertion

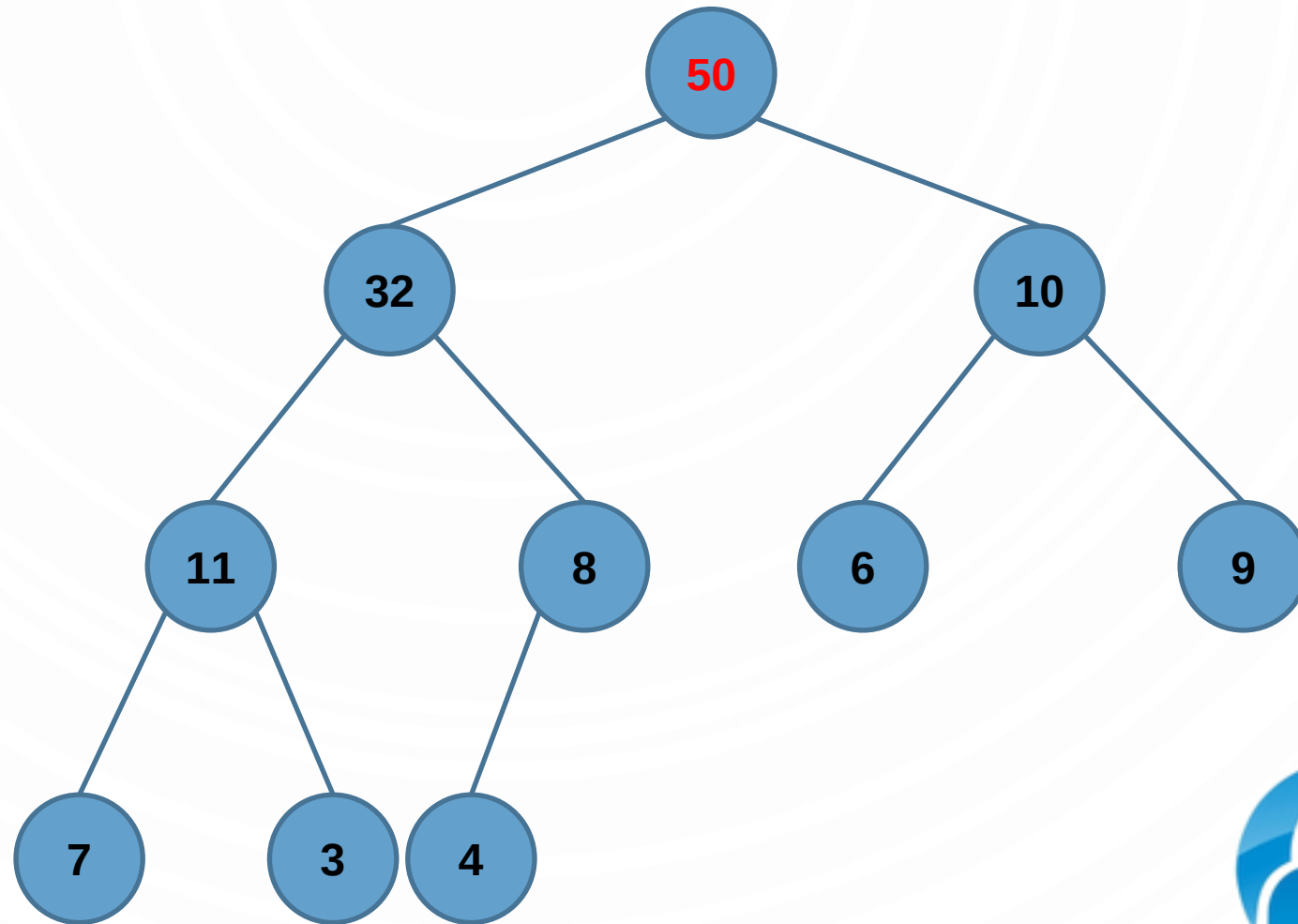
Pseudocode:

1. Insert the new node in the first valid (available) location;
2. while(new element doesn't comply with the heap restriction)
 switch between the new and the parent node;

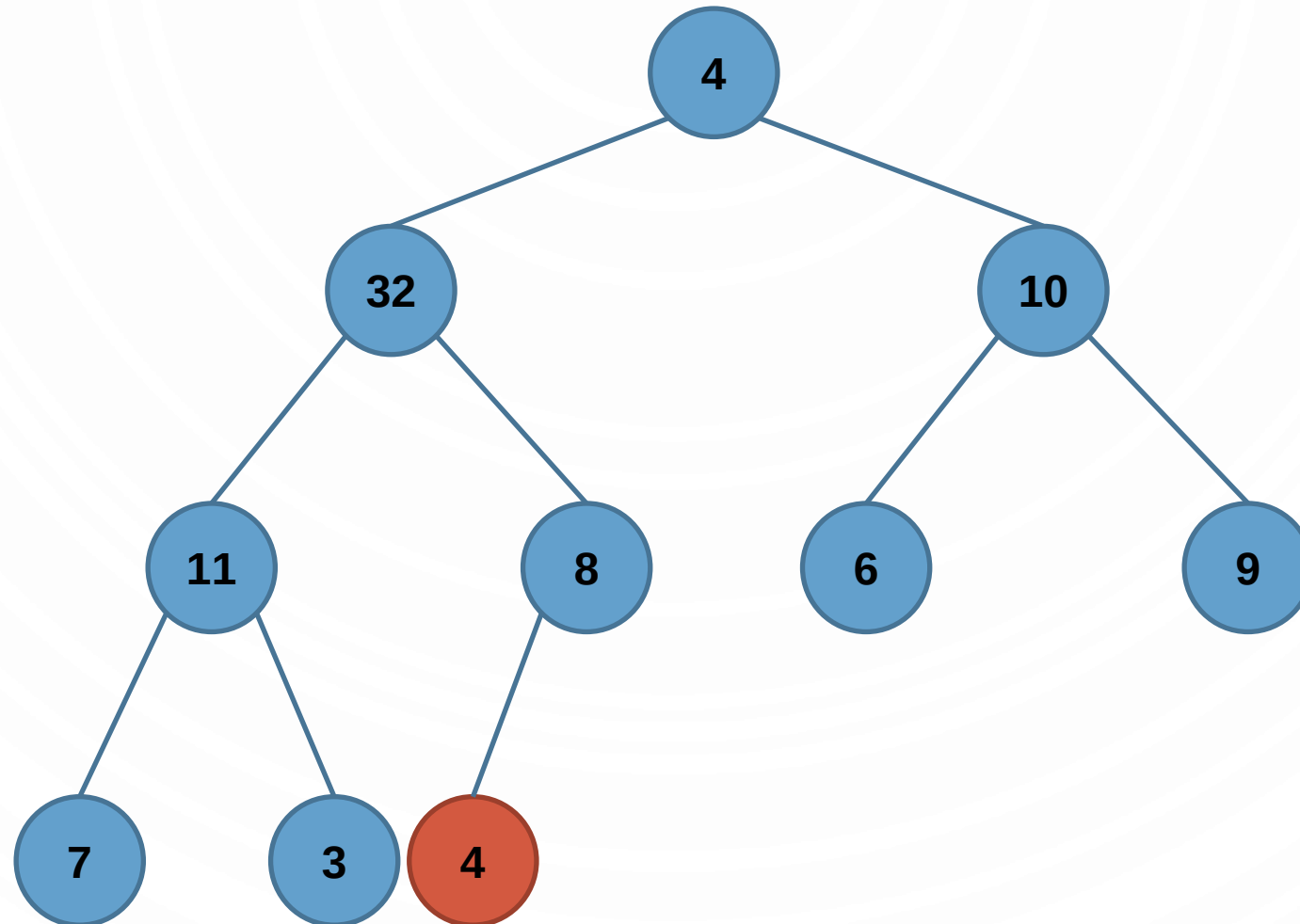


Heap - Deletion

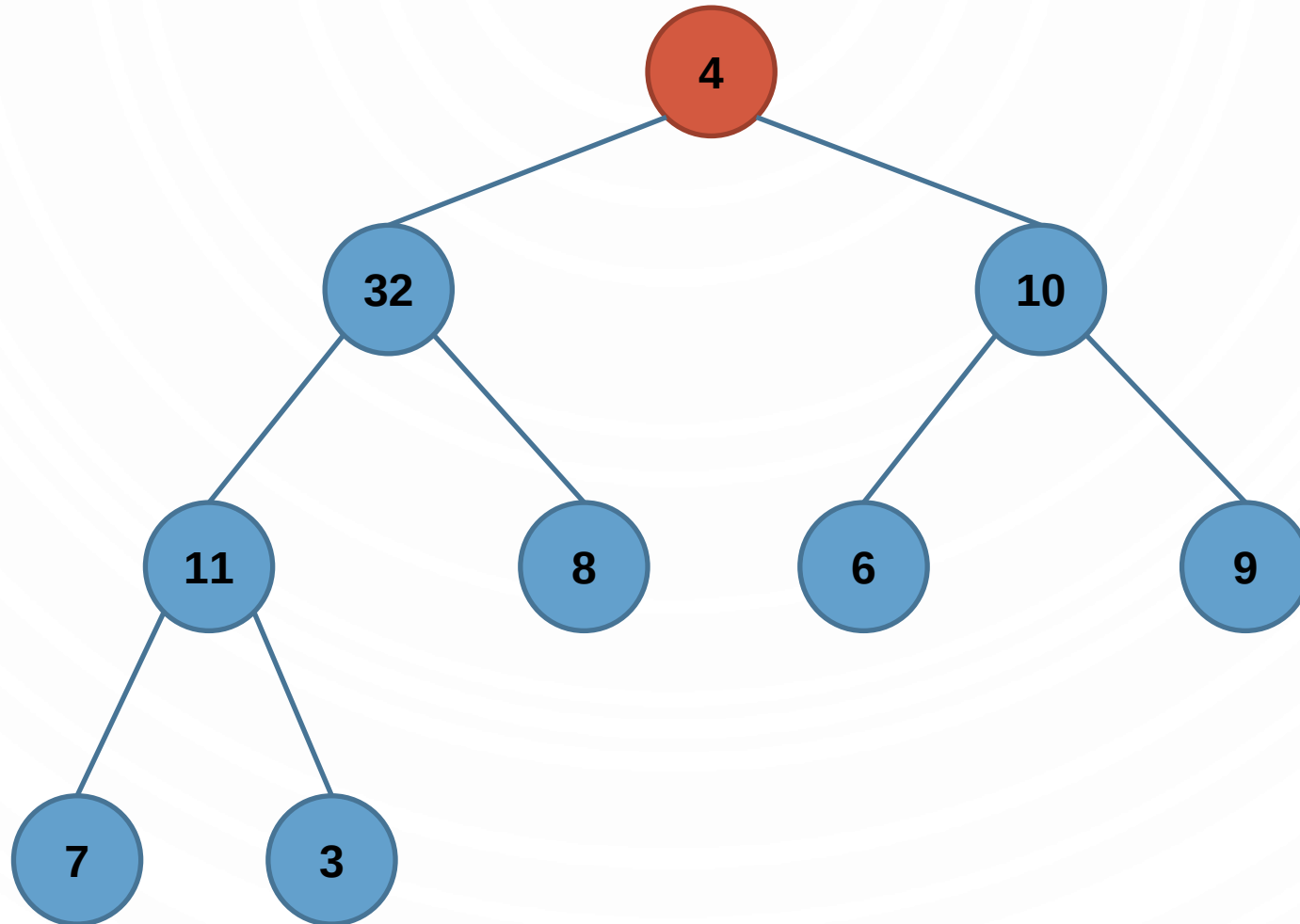
Delete 50



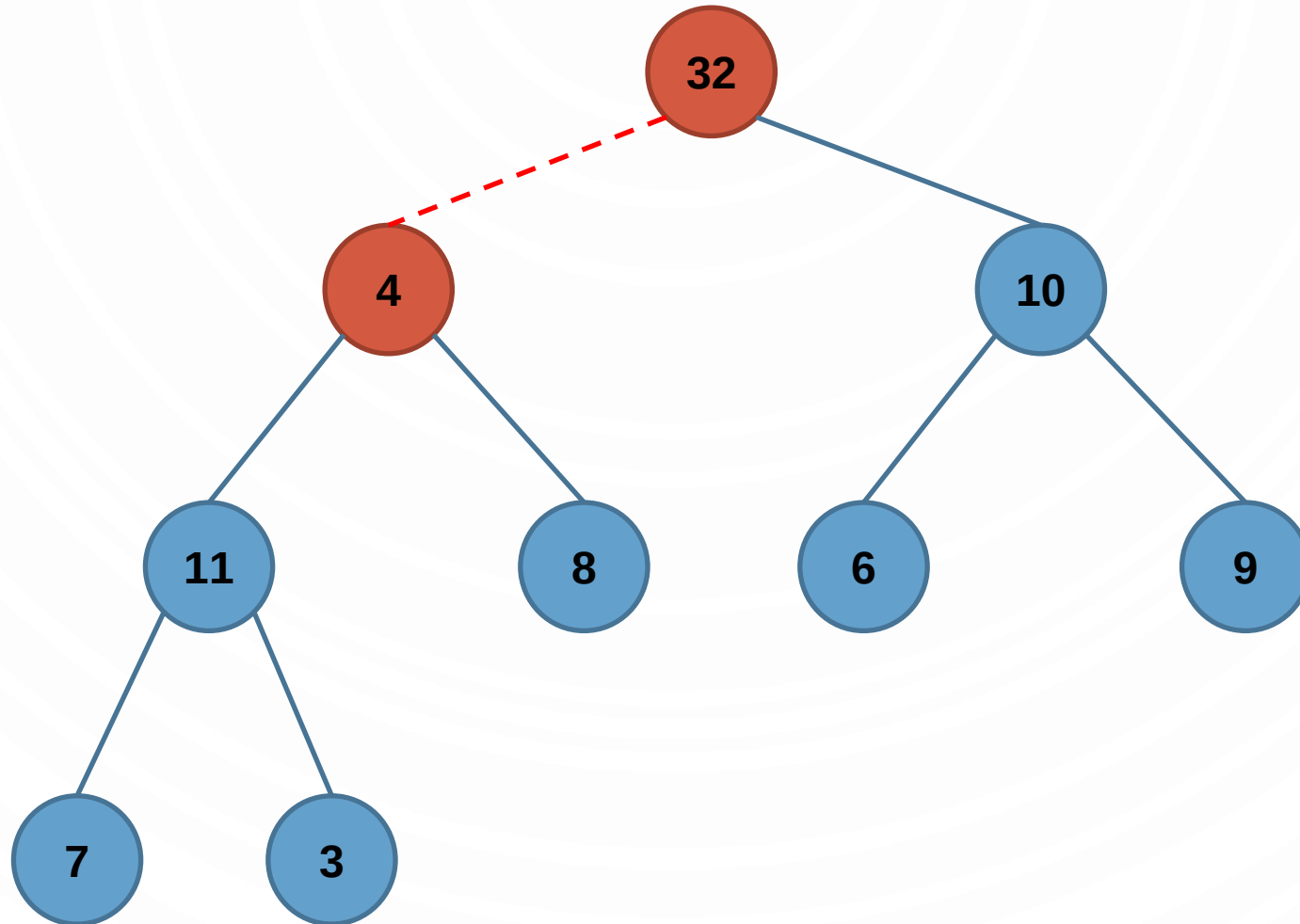
Heap - Deletion



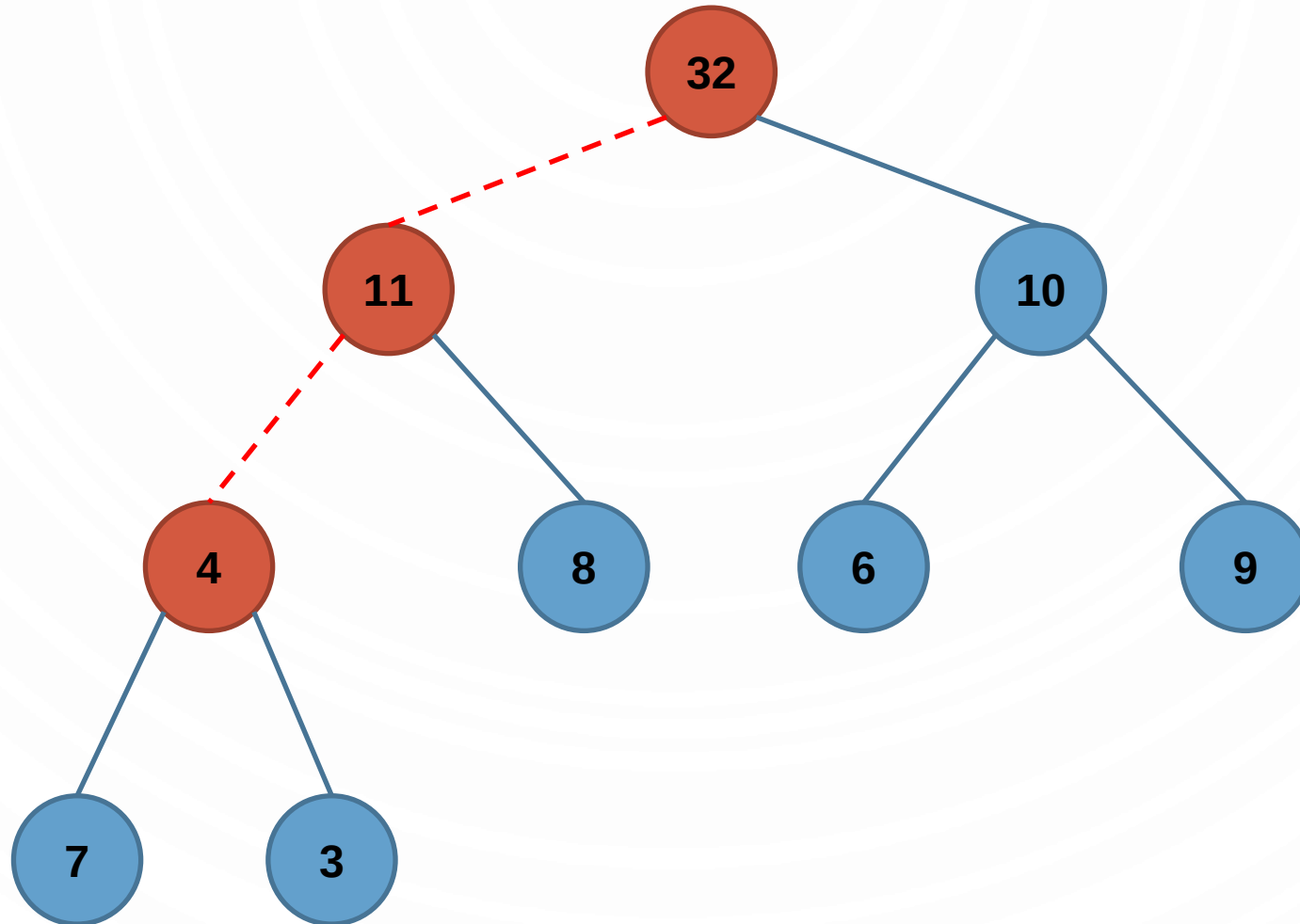
Heap - Deletion



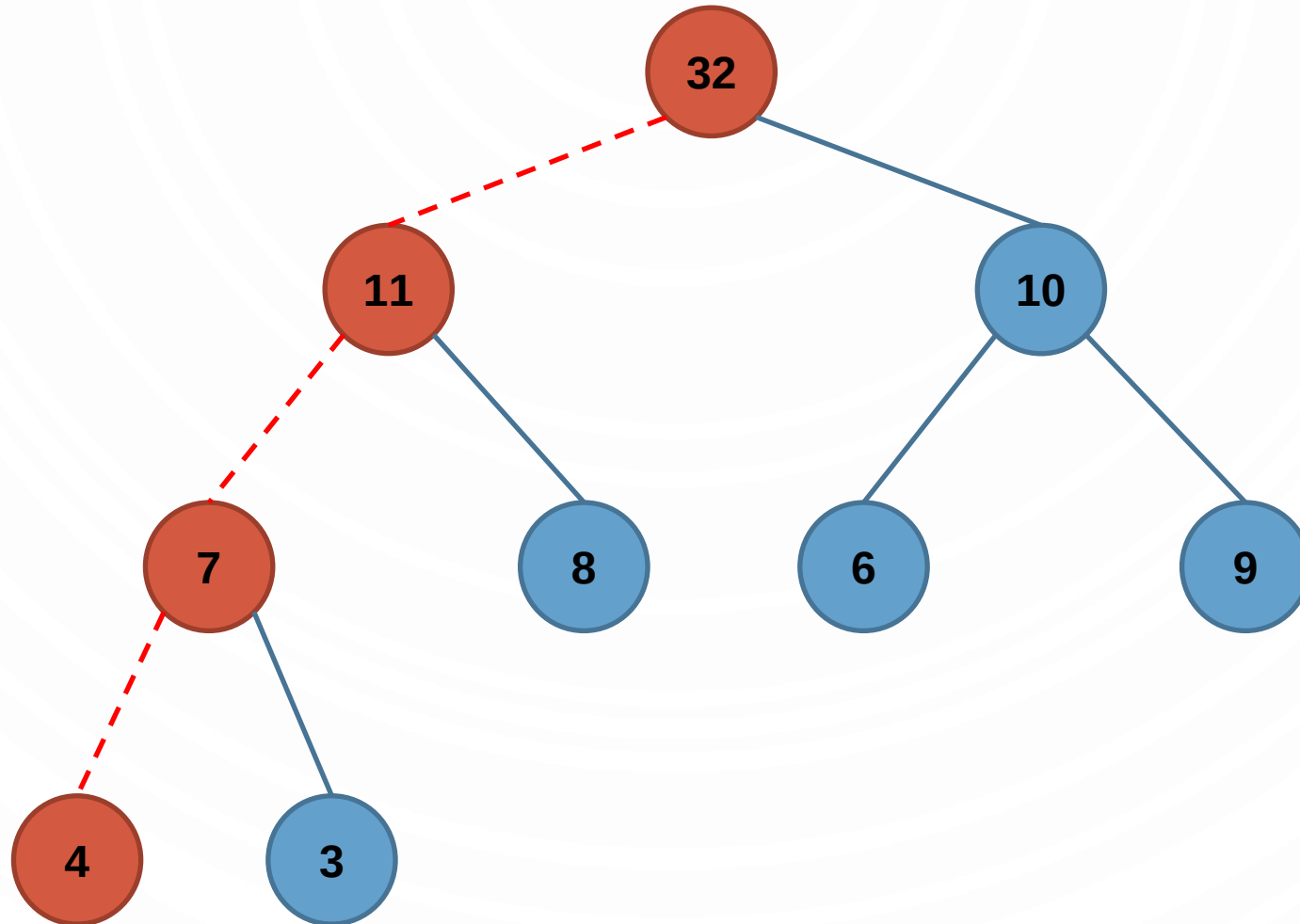
Heap - Deletion



Heap - Deletion



Heap - Deletion



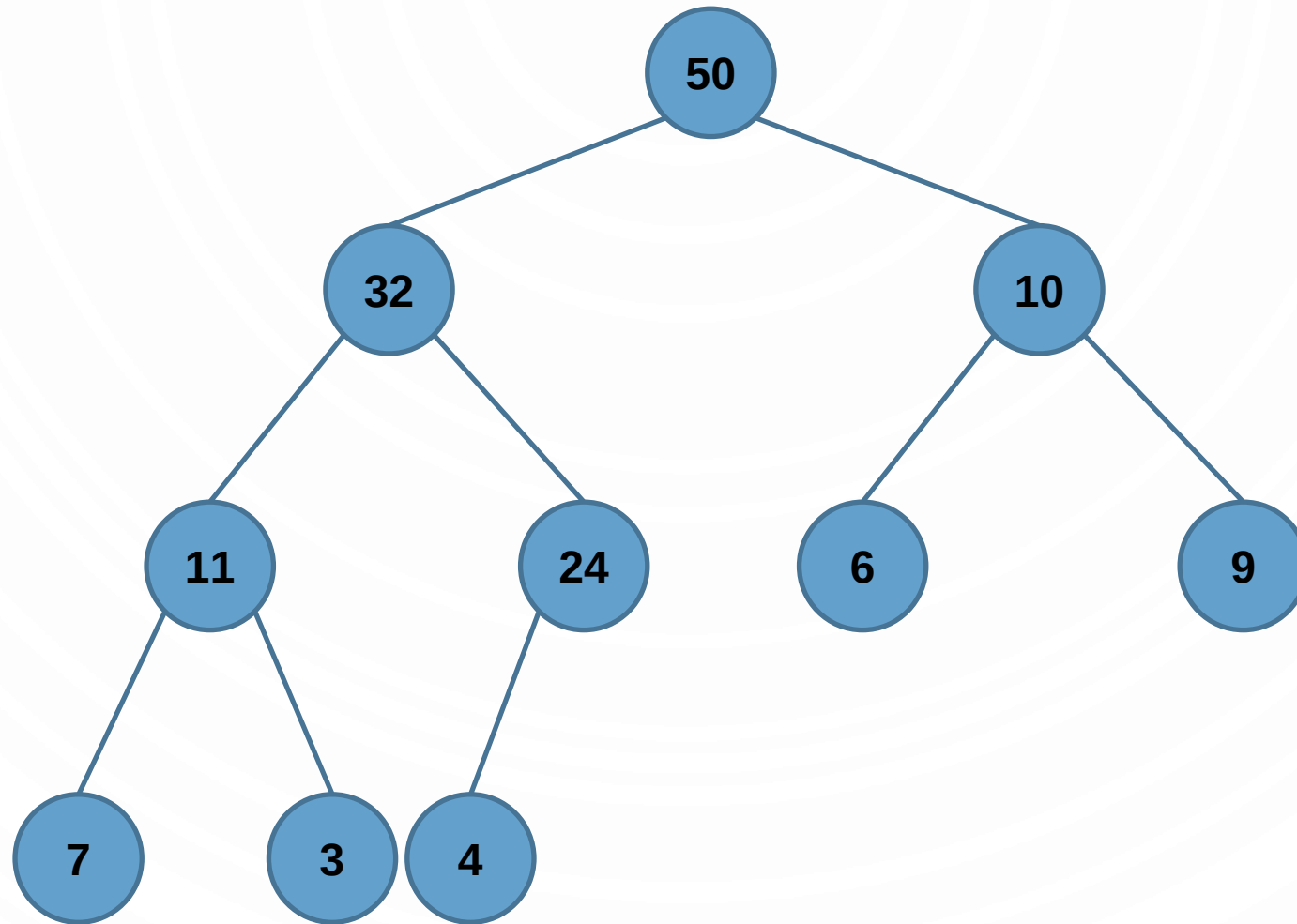
Heap - Deletion

Pseudocode:

1. saving the value of the root, the deleted node;
2. replace value of the root with the last element from the structure;
3. delete last element from the structure;
4. while(the new root doesn't comply with the heap condition)
 switch between the new root and the highest or lowest child node;



Heap Sort



Priority Queues - PQs



Insertion = re-heapification upward

Deletion = re-heapification downward



PQ Operations

- $\text{Insert}(S, x)$ – insert element x into set S
- $\text{Max}(S)$ – return element of S with the highest key
- $\text{Extract}(S)$ – min or max, return element x and remove it from the set
- $\text{increasePrio}(S, x, k)$ – increase the value of x key from the set at k



Priority Queues - PQs

Other operations:

- Increasing the priority of an element;
- Decreasing the priority of an element;
- Deleting an element;
- Searching for min;
- Searching for max;
- Heapsort;



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

